



PULPIT FLOW PRO

Product Requirements & Architecture Specification

Single Source of Truth | Version 1.0 | May 2026

PURPOSE

Church Production Platform

AUDIENCE

Development Team

This document is the authoritative reference for all design, development, and delivery decisions. No feature, architecture change, or UX deviation may occur without updating this document first.

SECTION 1 — EXECUTIVE OVERVIEW

1. Executive Overview

1.1 Project Identity

Product Name	Pulpit Flow Pro
Short Name	PFP
Project Type	Electron Desktop Application
Target User	Church AV/Tech teams, worship leaders, pastors
Platform	Windows 10/11, macOS 12+
License	Internal / Proprietary (church use)
Version	1.0 (Initial Release)
Reference Project	Bible Song Pro OBS (github.com/Johnbatey/bible-song-pro-obs)

1.2 Vision Statement

VISION	Pulpit Flow Pro is a professional-grade, all-in-one church production platform that replaces disconnected tools with a single Electron desktop application. It serves every screen in the building — congregation display, stage monitor, OBS/vMix stream overlay, and pastor confidence monitor — all controlled from one operator dashboard and driven by a real-time WebSocket server.
--------	---

1.3 What Bible Song Pro Does (Baseline Reference)

Bible Song Pro is the open-source project this application improves upon. Understanding it prevents the developer from reinventing what already works.

- Two HTML files: a control panel and a display output
- Operator types Bible verses or song lyrics into the panel
- BroadcastChannel API pushes content to the display in real-time
- Display HTML file is loaded into OBS as a Browser Source
- Auto-retrieve lyrics feature
- Built entirely with HTML, CSS, and vanilla JavaScript — no framework
- Works only on the same machine (BroadcastChannel is browser-tab scoped)
- No persistence, no database, no multi-device support

CRITICAL

Pulpit Flow Pro must do everything Bible Song Pro does, and dramatically more. No feature regression is acceptable. Every Bible Song Pro capability is a minimum baseline, not a ceiling.

1.4 Core Differentiators

Feature	Bible Song Pro (Reference)	Pulpit Flow Pro (Target)
Architecture	Two HTML files	Electron desktop app + local server
Communication	BroadcastChannel (same browser)	WebSocket (any device on network)
OBS Integration	Open HTML file directly	localhost URL — stable & multi-output
vMix Integration	Not supported	Full support via browser input
Outputs	One screen	4 independent output URLs
Remote Control	Same machine only	Any phone/tablet on WiFi
Song Storage	Session only (lost on close)	SQLite database — persists forever
Service Planning	None	Full order of service builder
Bible Translations	Limited	Multiple (KJV, NIV, ESV, NKJV, AMP, NLT)
AI Features	None	Claude API — scripture suggest, auto-format
Multi-Output	None	4 simultaneous independent displays
Stage Monitor	None	Dedicated worship team view
Pastor Monitor	None	Confidence monitor with private notes

SECTION 2 — SYSTEM ARCHITECTURE

2. System Architecture

2.1 Architecture Overview

Pulpit Flow Pro is a single Electron application that embeds a Node.js/Express HTTP server and a WebSocket server. All output screens are served as local web pages. OBS, vMix, secondary monitors, and remote devices connect to these URLs.

RULE

The Electron app is the single source of truth for all state. No output screen holds its own state. All screens are dumb renderers that receive instructions over WebSocket and display them.

2.2 Output URL Architecture

Each output is an independent URL served by the embedded Express server on localhost port 3000. These URLs are added to OBS/vMix as Browser Sources or to secondary monitors as web pages.

http://localhost:3000/display	Main congregation screen — full-screen lyrics, scripture, announcements
http://localhost:3000/stream	OBS/vMix stream overlay — transparent background, lower-third or full graphic mode
http://localhost:3000/stage	Stage monitor — lyrics + upcoming slide preview for worship team
http://localhost:3000/preacher	Confidence monitor — private pastor notes, current text, service position
http://localhost:3000/remote	Phone/tablet remote control — advances slides, controls service flow
http://localhost:3000/preview	Tech team preview — shows what is currently live on all outputs

2.3 How OBS Connects

In OBS Studio:

1. Sources → Add → Browser Source
2. Set URL to http://localhost:3000/stream (for overlay) or /display (for full takeover)
3. Width: 1920, Height: 1080
4. Uncheck "Shutdown source when not visible"
5. The display updates in real time whenever the operator sends content

2.4 How vMix Connects

In vMix:

- 6. Add Input → Web Browser
- 7. URL: `http://localhost:3000/stream`
- 8. Add to Overlay Channel 1–4 for lower-third use, or as a full input for takeover
- 9. vMix can transition it like any other input

2.5 Data Flow Diagram

[Operator Dashboard — Electron Window]

↓ Operator sends command (e.g., "display verse John 3:16 NIV")

[PFP Core Engine — Node.js in Electron Main Process]

- ↓ Looks up verse in local Bible database (SQLite)
- ↓ Formats payload as JSON
- ↓ Broadcasts via WebSocket to all connected clients

[WebSocket Clients]

- `/display` → Congregation screen renders full verse
- `/stream` → OBS overlay renders lower-third
- `/stage` → Stage monitor shows current + next
- `/preacher` → Confidence monitor shows verse + notes
- `/remote` → Remote control updates its status indicator

2.6 Technology Stack

Desktop Wrapper	Electron (latest stable) — packages app as .exe / .app
UI Framework	React 18 with Vite — operator dashboard and all output pages
Styling	Tailwind CSS — utility-first, fast, consistent
Local Server	Express.js — serves output HTML pages on localhost:3000
Real-Time Sync	ws (WebSocket library) — instant push to all connected screens
Database	SQLite via better-sqlite3 — song library, service plans, settings
Bible Data	Local JSON files — KJV, NIV, ESV, NKJV, AMP, NLT (no internet required)
AI Features	Anthropic Claude API — scripture suggestion, auto-format, sermon slides
Package Manager	npm
Build / Installer	electron-builder — creates Windows .exe installer and macOS .dmg
State Management	Zustand — lightweight React state manager
Icons	Lucide React

SECTION 3 — FEATURE MODULES

3. Feature Modules

Pulpit Flow Pro is organized into eight feature modules. Each module is independent but communicates through the shared WebSocket state layer. Modules are built in phase order (see Section 5).

Module 1 — Bible Engine

PURPOSE	Enable operators to find and display any Bible verse instantly, in any translation, without internet access.
---------	--

Requirements

- All 66 books of the Bible stored locally in SQLite from JSON source files
- Supported translations: KJV, NIV, ESV, NKJV, AMP, NLT — all loaded at install time
- Reference lookup: operator types "John 3:16" and it resolves instantly
- Full-text keyword search across entire Bible
- Side-by-side dual translation display — operator picks two translations; both show on screen
- Verse range support: "Romans 8:28-30" displays all three verses as one block
- Pastor Study Mode: shows current verse with private commentary/notes on preacher monitor only
- Display output shows: verse text, book + chapter + verse reference, translation abbreviation
- Works 100% offline — no API calls for verse lookup

Bible Data Source

Use publicly available open-license Bible JSON datasets. These are converted to SQLite tables at build time. The developer must verify the license of each translation before including it. KJV is public domain. NIV, ESV, NLT require license review.

Module 2 — Song Library

PURPOSE	Give the church a permanent, searchable song database that operators can use to display lyrics slide-by-slide during worship.
---------	---

Requirements

- SQLite database stores all songs with: title, artist, album, CCLI number, key, tempo, tags, year
- Lyrics stored as structured sections: Verse 1, Chorus, Verse 2, Bridge, Outro, etc.
- Operator can create, edit, duplicate, and delete songs from the dashboard

- Import songs from: plain text (with section labels), Planning Center (if API available), SongSelect (copy-paste)
- Song search by title, artist, CCLI number, or lyrics keyword
- Section locking: admin can lock a section order so operators cannot skip or reorder slides
- Chord chart view: a separate display mode shows chords above lyrics — outputtable to stage monitor
- Next-slide preview: operator sees upcoming section while current is live
- Auto-retrieve lyrics: operator types song name, app fetches lyrics from web (requires internet)
- Songs tagged with themes: Worship, Communion, Offering, Opening, Closing

Module 3 — Service Planner

PURPOSE	Allow the church to plan the entire Sunday service in advance and execute it sequentially without searching during the service.
---------	---

Requirements

- Service plan is an ordered list of items called blocks
- Block types: Welcome Slide, Song, Scripture, Sermon, Offering, Prayer, Announcement, Video, Custom Slide, Blank
- Drag-and-drop reordering of blocks before and during service
- Each block can be pre-configured (e.g., Song block already has "Way Maker" selected)
- Service templates: save a typical Sunday plan as a template and reuse week to week
- Operator clicks Next Block to advance — the correct content loads automatically
- Current block is highlighted in the plan; completed blocks are marked
- Service plan saved with date — historical plans are searchable
- Export/import service plans as .pfp files to share between team members
- Print-friendly service order for paper bulletins

Module 4 — Multi-Output Display Engine

PURPOSE	Drive four independent screens simultaneously, each with a different layout and purpose, all from one control action.
---------	---

Output Specifications

/display	Full-screen congregation view. Full background, large text. Used on projector or main screen.
/stream	OBS/vMix overlay. Transparent background by default. Lower-third or full-screen mode. Used on livestream.
/stage	Worship team stage monitor. Shows current lyrics + next section preview. Chord chart toggle.

/preacher	Pastor confidence monitor. Shows current content + pastor private notes + service position indicator.
-----------	---

Requirements

- Each output is an independent HTML page served via Express
- All outputs receive WebSocket messages and render their own layout from the same data
- Clearing the display sends a "blank" state — all outputs go dark simultaneously
- Operator can "freeze" one output while advancing another (e.g., keep congregation display on a verse while changing stage view)
- /stream output has a "lower-third mode" toggle: content appears only in the bottom 25% of the screen
- /stream background is transparent (rgba(0,0,0,0)) by default for OBS compositing
- All outputs support custom CSS themes independently

Module 5 — Theme & Branding Engine

PURPOSE	Make every output look professional and on-brand for the church without requiring design skills.
---------	--

Requirements

- Church name, logo, primary color, and secondary color are set once in Settings and applied globally
- Built-in themes: Worship (dark), Scripture (light), Sermon (neutral), Announcement (bold), Blank (black)
- Operator can switch themes with one click during service
- Per-output theme override: /stream can use a different theme than /display
- Background types: solid color, static image, looping video (MP4), motion background library
- Font selector: operator picks from a curated set of broadcast-safe fonts
- Text size, shadow, and outline controls for readability adjustment
- Custom themes can be saved and named (e.g., "Easter 2026 Theme")
- Theme manager: upload, preview, delete background images and videos

Module 6 — Announcements & Media Module

PURPOSE	Handle pre-service loops, event announcements, videos, and giving info without switching to another application.
---------	--

Requirements

- Announcement slide builder: title, body text, image, background, duration
- Announcement playlist: ordered list of slides that auto-advance

- Countdown timer slide: "Service begins in X minutes" with live countdown
- Pre-service loop mode: run announcement playlist automatically before service starts
- Giving/QR code slide: display church giving link as QR code + text
- Social media slide: display Instagram/Facebook handle with icon
- Video block: operator selects a video file; it plays in the display window
- Video controls: play, pause, stop, seek — operated from dashboard without touching display
- Lower-third announcement: text scrolls or appears at bottom of screen while other content is live

Module 7 — Network & Remote Control

PURPOSE	Allow worship leaders, pastors, and assistants to control or monitor the presentation from any device on the church WiFi.
---------	---

Requirements

- Remote control page at <http://localhost:3000/remote> — accessible on any phone/tablet browser on same WiFi
- Remote shows: current service block, current slide content, Prev / Next buttons
- Worship leader remote mode: can advance song sections only (not full operator control)
- Pastor remote mode: can advance sermon points, trigger "Next Scripture" — cannot change themes or songs
- Admin can enable/disable remote control and set PIN for access
- Live preview page at <http://localhost:3000/preview> — shows tech team what is live right now
- Network discovery: app displays its IP address on screen for easy device connection
- Multi-operator: second operator on another device can shadow the main operator (view-only)

Module 8 — AI Integration (Claude-Powered)

PURPOSE	Give operators intelligent assistance that reduces manual work and improves content quality.
---------	--

Requirements

- Powered by Anthropic Claude API (model: claude-sonnet-4-20250514)
- API key stored in app settings — operator provides their own key
- Scripture Suggester: operator types a sermon topic; AI suggests 3–5 relevant Bible verses
- Sermon-to-Slides: operator pastes sermon outline; AI formats it into presentation-ready point slides
- Lyric Cleaner: imported song lyrics are auto-formatted (correct capitalization, fix line breaks, remove extra spaces)
- Related Passage Finder: given an open passage, AI suggests thematically related scriptures
- Announcement Writer: operator types event details; AI generates a clean announcement slide text

- AI features are optional and gracefully disabled when API key is not set
- All AI calls happen server-side in the Electron main process — never exposed to browser

SECTION 4 — OPERATOR DASHBOARD UI

4. Operator Dashboard

4.1 Layout

The operator dashboard is the main Electron window. It is divided into four persistent zones:

Top Bar	App name, service status indicator (Live / Standby), active theme name, settings button
Left Panel	Service plan (current block list). Click any block to jump to it.
Center Panel	Active content area — changes based on current block type (Song, Scripture, Announcement, etc.)
Right Panel	Live preview thumbnails of all four output screens. Shows what each audience sees right now.
Bottom Bar	Global transport controls: Prev Slide, Next Slide, Blank All, Clear, Go Live toggle

4.2 Go Live Toggle

RULE	"Go Live" is a master switch. When OFF, operators can prepare and preview content freely without it appearing on any output. When switched ON, the current content is pushed to all outputs immediately. This prevents accidental display of incomplete slides.
------	---

4.3 Blank All Button

RULE	Blank All immediately sends a blank state to all outputs. The congregation screen, stream overlay, and stage monitor all go to a solid black (or theme-colored) screen. This is the emergency button. It must be one click, always visible, never hidden behind a menu.
------	---

4.4 Song Control View

When a Song block is active in the service plan, the center panel shows:

- Song title and artist
- Section list: Verse 1, Chorus, Verse 2, Bridge, etc. — clickable to jump
- Current section text displayed large
- Next section preview displayed smaller below
- Prev Section / Next Section navigation buttons
- Chord chart toggle button (sends chord view to /stage output)

4.5 Scripture Control View

When a Scripture block is active:

- Translation selector dropdown
- Book / Chapter / Verse pickers (cascading dropdowns)
- Or: direct text input field for typing references like "Rom 8:28"
- Verse range toggle: display single verse or a range
- Second translation toggle for dual-translation display
- Preview of how text will render on display screen

4.6 Keyboard Shortcuts

Space / Right Arrow	Next slide
Left Arrow	Previous slide
B	Blank all outputs
L	Toggle Go Live
Escape	Clear active content (go to standby slide)
Ctrl + F	Open Bible search
Ctrl + S	Save current service plan
F11	Toggle fullscreen dashboard

SECTION 5 — BUILD PHASES & DELIVERY

5. Build Phases

Development is divided into four phases. Each phase produces a working, testable build. No phase is skipped. Phase 1 must be complete and stable before Phase 2 begins.

Phase	Name	Modules	Priority
1	Foundation	Electron shell, Express + WebSocket server, SQLite setup, Bible database (KJV), basic display output (/display), basic operator panel with Go Live and Blank All	Critical
2	Core Production	Song library (full CRUD), Service planner, /stream output (OBS/vMix overlay), /stage output, all keyboard shortcuts, theme switching	Critical
3	Full Platform	/preacher output, /remote control, announcement builder, video playback, multi-translation Bible, motion backgrounds, custom themes	High
4	Intelligence Layer	AI integration (Claude API), sermon-to-slides, scripture suggester, lyric cleaner, Planning Center import, CCLI number tracking, print service order	Medium

5.1 Phase 1 Exit Criteria

Phase 1 is complete only when ALL of the following are true:

- Electron app launches without error on Windows 10 and macOS 12
- Express server starts on localhost:3000 on app launch
- http://localhost:3000/display renders in OBS Browser Source correctly at 1920x1080
- Operator types a Bible reference; verse appears on /display in under 500ms
- Blank All button instantly clears /display
- Go Live toggle works correctly — content does not appear on /display when Off
- App closes cleanly — server shuts down, no port remains occupied

5.2 Phase 2 Exit Criteria

- Song library persists across app restarts (SQLite confirmed working)
- /stream URL shows transparent-background overlay in OBS
- Service planner runs a full mock Sunday service start to finish without errors
- /stage shows next-section preview correctly
- All keyboard shortcuts function correctly

5.3 Phase 3 Exit Criteria

- /preacher output accessible on a phone browser on the same WiFi
- /remote advances slides correctly on a phone
- Announcement playlist auto-advances
- Motion background video loops behind lyrics without stuttering

5.4 Phase 4 Exit Criteria

- Scripture suggester returns relevant verses for a sermon topic via Claude API
- Sermon-to-slides produces at least 3 usable point slides from a 5-point outline
- AI features gracefully disabled and clearly labeled when API key is not set

SECTION 6 — NON-NEGOTIABLE RULES

6. Non-Negotiable Rules

These rules are absolute. No exception, no workaround, no "we can add it later." If any rule cannot be met in a given phase, development stops and the issue is escalated.

RULE 1

The app must work 100% offline for all core functions. Bible lookup, song display, service planning, and all output screens must function with no internet connection. Only AI features and auto-retrieve lyrics require internet.

RULE 2

OBS and vMix integration must work on Day 1 of Phase 1. The /display URL must load in OBS Browser Source and update in real time. This is non-negotiable because it is the primary use case.

RULE 3

Blank All is always one click. It is always visible on screen. It cannot be hidden behind a menu, a confirmation dialog, or a secondary panel. Ever.

RULE 4

No content appears on any output screen until the operator explicitly activates Go Live. Pre-loading, editing, and previewing are always safe operations with zero display impact.

RULE 5

The Express server must shut down cleanly when the app closes. Port 3000 must not remain occupied after the app exits. Implement proper cleanup in the Electron will-quit handler.

RULE 6

The app must not require admin privileges to run. Church AV volunteers should be able to launch it from the desktop like any normal app.

RULE 7

All user data (songs, service plans, settings) is stored locally on the machine in SQLite. No cloud sync, no user accounts, no external database. The church owns their data entirely.

RULE 8

The Claude API key is stored in local app settings only. It is never logged, never transmitted except to the Anthropic API endpoint, and never stored in plain text in a config file that might be shared.

RULE 9

Response time from operator action to display update must be under 500ms in all cases on a standard local network. This is a live production environment. Lag is unacceptable.

RULE 10

The product name is Pulpit Flow Pro in all UI text, documentation, window titles, and installer names. No abbreviation in user-facing text except PFP in compact contexts.

SECTION 7 — DATA MODELS

7. Data Models (SQLite Schema)

All persistent data is stored in a single SQLite file located in the Electron app's userData directory. The schema below defines every table required for Phase 1 and Phase 2.

7.1 songs

id	INTEGER PRIMARY KEY AUTOINCREMENT
title	TEXT NOT NULL
artist	TEXT
ccli_number	TEXT
key_sig	TEXT
tempo	TEXT
tags	TEXT (JSON array as string)
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME

7.2 song_sections

id	INTEGER PRIMARY KEY AUTOINCREMENT
song_id	INTEGER (FK → songs.id)
type	TEXT — "verse", "chorus", "bridge", "pre-chorus", "outro", "custom"
label	TEXT — e.g. "Verse 1", "Chorus"
lyrics	TEXT
chords	TEXT
position	INTEGER — sort order within song
locked	BOOLEAN DEFAULT 0

7.3 service_plans

id	INTEGER PRIMARY KEY AUTOINCREMENT
title	TEXT NOT NULL
service_date	DATE

template	BOOLEAN DEFAULT 0
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP

7.4 service_blocks

id	INTEGER PRIMARY KEY AUTOINCREMENT
plan_id	INTEGER (FK → service_plans.id)
type	TEXT — "song", "scripture", "sermon", "announcement", "video", "blank", "custom"
position	INTEGER — order in plan
song_id	INTEGER nullable (FK → songs.id)
scripture_ref	TEXT nullable — e.g. "John 3:16 NIV"
custom_data	TEXT nullable — JSON for custom or announcement blocks
notes	TEXT — operator notes for this block
completed	BOOLEAN DEFAULT 0

7.5 settings

key	TEXT PRIMARY KEY
value	TEXT

Settings keys include: church_name, church_logo_path, primary_color, secondary_color, default_translation, claude_api_key (encrypted), server_port, remote_pin.

SECTION 8 — WEBSOCKET MESSAGE PROTOCOL

8. WebSocket Message Protocol

All real-time communication between the Electron main process and output screens uses this standardized JSON message format. Every message has a type and a payload.

8.1 Message Envelope

`{ "type": "MESSAGE_TYPE", "payload": { ... }, "timestamp": 1234567890 }`

8.2 Message Types

DISPLAY_VERSE	payload: { text, reference, translation, secondText?, secondTranslation?, theme }
DISPLAY_LYRICS	payload: { songTitle, artist, sectionLabel, lines[], nextSectionLabel?, nextLines?, theme }
DISPLAY_SLIDE	payload: { title?, body?, background?, type: "announcement" "custom" "blank" }
DISPLAY_BLANK	payload: {} — blanks all outputs immediately
DISPLAY_CLEAR	payload: {} — returns all outputs to standby/logo screen
THEME_CHANGE	payload: { themeName, primaryColor, secondaryColor, backgroundUrl? }
SERVICE_ADVANCE	payload: { blockIndex, blockType, blockTitle } — for stage/preacher awareness
GO_LIVE	payload: { live: true false }
STAGE_CHORD_MODE	payload: { enabled: true false }
PREACHER_NOTE	payload: { note } — private text for preacher monitor only
PING	payload: {} — keepalive; clients respond with PONG

RULE	All output screens must handle any unknown message type gracefully — log it and ignore it. Never crash on an unrecognized message type. The protocol will evolve and older output pages must remain stable.
------	---

SECTION 9 — DESIGN LANGUAGE

9. Design Language

9.1 Operator Dashboard Design Principles

- Dark theme by default — church AV booths are typically dimly lit
- High contrast — operators must read the UI in low light at a glance
- Minimal clicks — the most common actions (Next, Blank, Go Live) are always one click
- No modal dialogs during live service — nothing that blocks the operator's view
- Status is always visible — Go Live state, current block, and server status are always in the top bar

9.2 Output Screen Design Principles

- Congregation screen: large, readable text with strong contrast. Minimum 64px font size for lyrics. Minimum 48px for verse text.
- Stream overlay: text must be readable over any video background. Always use text shadow or outline.
- Stage monitor: functional over beautiful. Show current + next clearly. Chord chart must be legible from 10 feet away.
- Preacher monitor: dark background, large text, minimal distraction. Service position indicator at top.

9.3 Default Color Palette

Primary Dark	#1B2A4A — Navy — backgrounds, headers
Accent Gold	#C9A84C — Gold — highlights, active states, dividers
Action Blue	#2E5FA3 — Blue — buttons, links, interactive elements
Light Surface	#EEF3FB — Light Blue-White — content panels
Danger	#C0392B — Red — Blank All button, warnings
Success	#1E7E34 — Green — Go Live active state, connected indicators
Text Primary	#FFFFFF — White — primary text on dark backgrounds
Text Secondary	#AAAAAA — Light Gray — secondary labels, timestamps

9.4 Typography

Dashboard UI	Inter or System UI — clean, readable, modern
--------------	--

Display Headings	Montserrat Bold or Playfair Display — worship feel
Display Body	Open Sans or Noto Sans — maximum legibility at scale
Monospace (code)	JetBrains Mono — for any debug or settings panels

SECTION 10 — PROJECT FILE STRUCTURE

10. Project File Structure

The following is the canonical directory structure. Developers must not deviate without documenting the reason in a PR description.

pulpit-flow-pro/	Project root
├─ electron/	Electron main process files
└─ main.js	App entry point, window management, server startup
└─ server.js	Express + WebSocket server
└─ database.js	SQLite connection and query helpers
└─ bible.js	Bible lookup logic
└─ ai.js	Claude API integration
├─ src/	React frontend (operator dashboard)
└─ App.jsx	Root component with layout zones
└─ components/	Reusable UI components
└─ panels/	Left/Center/Right panel components
└─ views/	Song view, Scripture view, Announcement view, etc.
└─ store/	Zustand state stores
├─ outputs/	Output screen HTML/React pages
└─ display.html	Congregation screen
└─ stream.html	OBS/vMix overlay
└─ stage.html	Stage monitor
└─ preacher.html	Confidence monitor
└─ remote.html	Phone remote control
└─ preview.html	Tech team preview
├─ data/	Bible JSON source files by translation
├─ assets/	Logos, default backgrounds, motion backgrounds
├─ scripts/	Build scripts (import Bible data to SQLite, etc.)
├─ package.json	Dependencies and build config
└─ electron-builder.yml	Installer configuration for Windows/macOS

SECTION 11 — GLOSSARY & REFERENCE

11. Glossary

PFP	Pulpit Flow Pro — the application
Operator	The AV tech team member running the software during service
Block	A single item in the service plan (Song, Scripture, Announcement, etc.)
Go Live	The master switch that enables output to all display screens
Blank All	Emergency button that sends black screen to all outputs instantly
Output Screen	Any of the four web pages served at localhost:3000/[path]
WebSocket	Persistent two-way connection used for real-time display updates
BroadcastChannel	Browser API used by Bible Song Pro (same-tab only — replaced by WebSocket in PFP)
Browser Source	OBS feature that renders a web URL as a video source layer
Lower-Third	On-screen graphic appearing in the bottom portion of a video frame
Stage Monitor	Screen facing the worship team on stage, showing lyrics/chords
Confidence Monitor	Screen visible only to the preacher, showing current content + notes
CCLI	Christian Copyright Licensing International — song licensing organization
SQLite	Embedded local database — no server required, single file on disk
Electron	Framework for building desktop apps using web technologies (HTML/CSS/JS)
better-sqlite3	Recommended synchronous Node.js SQLite library for Electron apps
Zustand	Lightweight React state management library
ws	Node.js WebSocket library used for the real-time push server

DOCUMENT AUTHORITY STATEMENT

This document is the single source of truth for Pulpit Flow Pro.

Any feature, architecture decision, naming convention, or UX pattern not explicitly described here must be brought back to the product owner before implementation. The developer must not infer intent, fill gaps independently, or deviate from this specification without written approval. When in doubt, ask.